

Rust Fundamentos

Notas de estudio

Giovanny Alfonso Chávez Cenicerros

Giovanny Alfonso Chávez Ceniceros

Rust Fundamentos

Notas de estudio

Contenido

1. Sintaxis y Semántica

- 1.1 Tokens
- 1.2 Tipos de Dato
- 1.3 Estructuras de Control
- 1.4 Tipos de Memoria
- 1.5 Funciones

2. Memory Safety

- 2.1 Ownership
- 2.2 Borrowing y Lifetimes

3. Enums, Option y Result

- 3.1 Enums
- 3.2 Option
- 3.3 Result

Contenido (cont.)

4. Estructuras de Datos

- 4.1 Vec
- 4.2 Iteradores
- 4.3 HashMap y HashSet
- 4.4 Structs
- 4.5 Listas Enlazadas
- 4.6 Árboles
- 4.7 Big-O (resumen)

5. Traits y Generics

- 5.1 Traits
- 5.2 Generics

Sintaxis y Semántica

Tokens

Palabras reservadas (*Keywords*)

1. Son la columna vertebral de la sintaxis de Rust.
 2. Dictan la definición de los bloques fundamentales de construcción de código.
 3. Omitir o usar incorrectamente alguna de ellas resulta en un error en tiempo de compilación.
- `let` se usa para la declaración de variables (auxiliada por `mut` para permitir la mutabilidad).
 - `fn` se usa para la definición de funciones (auxiliada por `return` para devolver valores explícitamente).
 - `if` se usa para condiciones (junto con `else` y `match` para bifurcar el flujo del programa).
 - `loop`, `while` y `for` se usan para ciclos (donde `in` auxilia a los ciclos `for`).
 - Existen muchas más (como `self`, `struct`, `enum`, `trait`, `impl`, `mod`, `type`), las cuales tienen nichos de uso más específicos o avanzados.

Identificadores

1. Son la parte personalizable del programa.
2. Es el nombre que el desarrollador asigna a un recurso (como una variable o función) para poder invocarlo más adelante.
3. Seguir las convenciones de nomenclatura de Rust (*snake_case*) y mantener nombres descriptivos mejora significativamente la legibilidad y el mantenimiento del código.

Literales

1. Son valores fijos que se incrustan directamente en el código fuente (como `42`, `"hola"` o `true`).
2. Son inmutables por naturaleza y ofrecen ventajas de rendimiento, ya que el compilador los conoce desde el principio.

Operadores

1. Son símbolos que indican al compilador la ejecución de tareas matemáticas, lógicas o de asignación.
 2. Actúan sobre variables y literales (llamados operandos) para producir un nuevo valor.
- `+`, `-`, `*`, `/`, `%` realizan operaciones aritméticas (suma, resta, multiplicación, división y residuo).
 - `==`, `!=`, `<`, `>`, `<=`, `>=` realizan comparaciones que devuelven un valor booleano (`true` o `false`).
 - `&&`, `||`, `!` ejecutan la lógica booleana (AND, OR y NOT).
 - `=` asigna valores, y puede combinarse con operadores aritméticos (como `+=` o `--`) para crear asignaciones compuestas.

Puntuaciones

1. Son caracteres individuales que definen la estructura semántica y el ritmo del código.
 2. Aunque parecen sutiles, son estrictamente obligatorios para que el compilador entienda dónde termina una idea y empieza otra.
- El punto y coma ; se usa para cerrar enunciados y expresiones de asignación.
 - Los dos puntos : se usan para especificar o anotar tipos de datos.

Delimitadores

1. Son caracteres que definen la estructura semántica del código respecto a los **ámbitos (scopes)** y a las secuencias de datos.
 2. **Siempre trabajan en pares** (apertura y cierre) para aislar o agrupar información.
- **Llaves** `{ }` : Definen un bloque de código y crean un ámbito (*scope*) aislado; todo lo declarado dentro muere al cerrarse la llave y no es accesible desde fuera.
 - **Corchetes** `[ ]` : Se utilizan para declarar arreglos (arrays) y para acceder a sus elementos mediante índices (ej. `let arr = [1, 2, 3];` y `arr[0]`).
 - **Paréntesis** `( )` : Encapsulan parámetros y argumentos en la declaración e invocación de funciones, y agrupan operaciones matemáticas para definir su prioridad.

Tipos de Dato

CATEGORÍA	TIPO	DESCRIPCIÓN	EJEMPLO / SINTAXIS
Numéricos	i8, i16, i32, i64, i128, isize	Enteros con signo (complemento a 2).	let x: i32 = -42; let ptr: isize = -1;
	u8, u16, u32, u64, u128, usize	Enteros sin signo.	let x: u8 = 255; let idx: usize = 0;
	f32, f64	Números de punto flotante según estándar IEEE 754.	let x: f64 = 3.1416; let y: f32 = 2.5;

CATEGORÍA	TIPO	DESCRIPCIÓN	EJEMPLO / SINTAXIS
Texto	char	Representa un carácter puro de cualquier alfabeto, emoji o símbolo.	<pre>let c: char = 'z'; let emoji = '🦀';</pre>
	str	Cadena de texto fija. Se utiliza casi siempre como referencia inline (<code>&str</code>).	<pre>let s: &str = "hola";</pre>
	String	Cadena de texto dinámica, mutable, de tamaño variable.	<pre>let s = String::from("hola"); let s2 = "mundo".to_string();</pre>

CATEGORÍA	TIPO	DESCRIPCIÓN	EJEMPLO / SINTAXIS
Lógicos y Vacíos	<code>bool</code>	Tipo booleano clásico de 1 byte de tamaño.	<pre>let activo: bool = true; let bandera = false;</pre>
	<code>()</code>	Equivale al concepto de <code>void</code> de otros lenguajes.	<pre>let vacio: () = (); fn log() -> () {}</pre>

CATEGORÍA	TIPO	DESCRIPCIÓN	EJEMPLO / SINTAXIS
Colecciones	<code>[T; N]</code>	Arreglo (<i>Array</i>). Colección homogénea de longitud fija conocida en tiempo de compilación (<code>N</code>).	<code>let arr: [i32; 3] = [1, 2, 3];</code>
	<code>(T, U, ...)</code>	Tupla. Colección heterogénea de longitud fija.	<code>let t: (i32, &str, bool) = (42, "hola", true);</code>
	<code>[T]</code>	Secuencia contigua de tamaño dinámico de tipos <code>T</code> . Se opera mediante <code>&[T]</code> .	<code>let sub_arr: &[i32] = &arr[1..3];</code>

CATEGORÍA	TIPO	DESCRIPCIÓN	EJEMPLO / SINTAXIS
Personalizados	<code>struct</code>	Estructuras. Permiten agrupar múltiples valores relacionados de distintos tipos bajo un mismo nombre.	<pre>struct Usuario { ... }</pre>

```
struct Person {  
    name: String,  
    age: u8,  
    is_dev: bool,  
}
```

```
fn main() {  
    let developer = Person {  
        name: String::from("Gio"),  
        age: 25,  
        is_dev: true,  
    };  
}
```

A diferencia de las tuplas, cada dato dentro de una `struct` tiene un **nombre claro (campo)**. Esto te permite modelar entidades del mundo real con un formato fijo que el compilador de Rust valida estrictamente.

Estructuras de Control

1. Condicional Estándar (`if` / `else`)

Evalúa una condición booleana pura. No lleva paréntesis en la condición.

```
if condicion_booleana {  
    // Se ejecuta si es true  
} else {  
    // Opcional: Se ejecuta si es false  
}
```

2. Coincidencia de Patrones Completa (`match`)

Obligatorio en Rust para evaluar múltiples caminos. Debe ser exhaustivo (cubrir todas las posibilidades).

```
match expresion {  
    patron_a => { /* código si coincide A */ },  
    patron_b => { /* código si coincide B */ },  
    _ => { /* opcional: "catch-all" para cualquier otro caso */ },  
}
```

3. Condicional de Patrón Único (`if let`)

La versión compacta de match cuando solo te importa un caso específico (ideal para extraer valores de Option o Result).

```
if let patron = expresion {  
    // Se ejecuta si coincidió; las variables internas del patrón se pueden usar aquí  
} else {  
    // Opcional: Se ejecuta si no coincidió  
}
```

4. Bucle por Colección (`for in`)

La forma más común y segura de iterar en Rust. Funciona con rangos, arreglos o vectores.

```
for variable in coleccion_o_rango {  
    // Se ejecuta por cada elemento (variable toma el valor actual)  
}
```

5. Bucle Condicional (`while`)

Ejecuta el bloque repetidamente mientras la condición booleana sea verdadera.

```
while condicion_booleana {  
    // Se ejecuta en bucle mientras sea true  
}
```

6. Bucle de Patrón Único (`while let`)

Ejecuta el bucle repetidamente mientras la expresión siga coincidiendo con el patrón (muy usado para vaciar colas o canales).

```
while let patron = expresion {  
    // Se ejecuta en bucle mientras siga coincidiendo el patrón  
}
```

7. Bucle Infinito (`loop`)

Un bucle eterno que no depende de una condición inicial. Es óptimo en Rust porque el compilador sabe que la única forma de salir es con un `break` explícito.

```
loop {  
    // Se ejecuta infinitamente  
    if condicion_de_salida {  
        break; // Detiene el bucle (puede devolver un valor: break valor;)  
    }  
}
```

Tipos de Memoria

El Stack

El Stack es una estructura de datos de acceso ultra rápido. Funciona bajo la regla de que lo último en llegar es lo primero en salir. El compilador solo permite guardar aquí datos cuyo tamaño exacto se conoce desde el momento en que se escribe el código.

- **¿Cómo funciona?:** Al llamar a una función, sus variables se "apilan" una arriba de otra en memoria. Al terminar la función, todo ese bloque se limpia instantáneamente.
- **Tipos que viven aquí:** Los Literales numéricos y booleanos, caracteres individuales (`char`), y las colecciones de tamaño fijo (Arreglos `[T; N]` y Tuplas).

- **Comportamiento:** Cuando una variable almacena un dato que vive completamente en el Stack, Rust permite que su valor se **duplique automáticamente** al asignarla a otra variable. Al hacer `let y = x;`, Rust clona los bits en la memoria de forma instantánea y transparente.
- **Resultado:** Ambas variables siguen vivas y son independientes. Esto ocurre porque duplicar datos pequeños y fijos en el Stack es sumamente barato para la computadora.

```
let x = 42;    // Vive en el Stack
let y = x;     // Se duplica automáticamente
println!("x: {}, y: {}", x, y); // ¡Ambos se pueden usar!
```

El Heap

El Heap se utiliza para datos más complejos cuyo tamaño es dinámico, desconocido al compilar, o que necesitan crecer y cambiar durante la ejecución del programa.

- **¿Cómo funciona?**: El sistema operativo busca un espacio libre en el Heap, guarda los datos ahí y genera un **apuntador** (una variable con la dirección de esa ubicación). El apuntador (que mide siempre lo mismo) se guarda en el Stack para acceso rápido, pero el contenido real está en el Heap.
- **Tipos que viven aquí**: Estructuras dinámicas como `String` o listas que pueden crecer.

- **Comportamiento:** A diferencia del Stack, los datos que se guardan en el Heap **no se duplican de forma automática**. Rust implementa una regla estricta de propiedad para evitar accidentes en la memoria. Al asignar una variable del Heap a otra (ej. `let s2 = s1;`), Rust no duplica el contenido del Heap; solo copia la "tarjeta/puntero" en el Stack y **destruye** la variable original (`s1`).
- **Resultado:** La propiedad del dato se **mueve** a la nueva variable. La variable vieja queda totalmente inválida para proteger el programa.

```
let s1 = String::from("hola"); // Datos en el Heap
let s2 = s1; // La propiedad se mueve a s2. s1 queda inválida.

// println!("{}", s1);
// ❌ ERROR de compilación: No puedes usar s1 porque su valor se movió.
```

Funciones

Punto de Entrada y Definición de Funciones

```
fn bienvenida(nombre: &str) {    // función auxiliar
    println!("¡Hola, {}!", nombre);
}

fn main() {                      // punto de entrada obligatorio
    bienvenida("Rust");
}
```

- `fn` declara una función; el nombre sigue convención *snake_case*.
- `main` es el único identificador reservado como punto de arranque.
- Las funciones pueden definirse antes o después de `main`; el orden no importa.
- Se invocan escribiendo su nombre seguido de `()`.

Parámetros y Valores de Retorno

```
fn sumar(a: i32, b: i32) -> i32 {  
    a + b // expresión sin `;` → valor de retorno implícito  
}  
  
fn main() {  
    let resultado = sumar(3, 5);  
    println!("{}", resultado); // 8  
}
```

- Cada parámetro requiere nombre y tipo separados por `:` — Rust no los infiere.
- `->` seguido del tipo declara qué devuelve la función.
- La última **expresión** sin `;` es el retorno implícito; agregar `;` la convierte en enunciado y causa un error de compilación.
- `return valor;` permite retornos explícitos o anticipados.

Programa Completo

```
fn cuadrado(n: i32) → i32 {  
    n * n  
}  
  
fn mensaje(nombre: &str) → String {  
    format!("¡Hola, {}!", nombre)  
}  
  
fn main() {  
    println!("{}", mensaje("Rust")); // ¡Hola, Rust!  
    println!("42 = {}", cuadrado(4)); // 42 = 16  
}
```

1. `main` orquesta el flujo general.
2. Las funciones auxiliares encapsulan tareas reutilizables.
3. Los valores fluyen mediante parámetros (entrada) y retornos (salida).

Memory Safety

Ownership

1. Todos los programas tienen que gestionar la forma en que utilizan la memoria de la computadora mientras se ejecutan.
 - Algunos lenguajes tienen un recolector de basura (*garbage collection*) que busca regularmente la memoria que ya no se utiliza a medida que el programa corre.
 - En otros lenguajes, el programador debe reservar y liberar la memoria de forma explícita.
2. Rust utiliza un tercer enfoque: la memoria se gestiona a través de un conjunto de reglas que el compilador verifica. Si se viola cualquiera de estas reglas, el programa no compilará.

El sistema de **Ownership** es un conjunto de reglas que gobiernan cómo un programa en Rust gestiona la memoria:

1. Cada valor tiene un propietario (*owner*).
2. Solo puede haber un propietario a la vez.
3. Cuando el propietario sale del ámbito (*scope*), el valor se destruye (*dropped*).

El *Ownership* (propiedad) define quién es responsable de la información en memoria, lo que naturalmente conlleva la capacidad de transferir esta responsabilidad. La propiedad de la información se transfiere a través de:

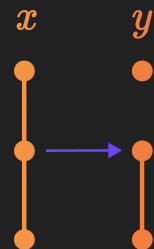
- **Asociaciones** (ej. `y = x;`)
- **Asignaciones** (ej. `let y = x;`)
- **Paso de parámetros** (enviar variables a una función)
- **Retornos** (devolver valores desde una función)

El comportamiento de los datos ante estos fenómenos variará dependiendo de dónde se almacene la información:

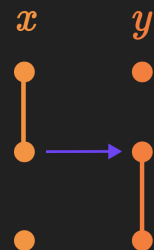
- **Si el dato vive en el Stack:** El valor se **duplica automáticamente**. Ambas variables se quedan con una copia independiente y siguen vivas.
- **Si el dato vive en el Heap:** Se realiza una **Transferencia (Move)**. La propiedad se mueve al nuevo destino y la variable original se destruye de inmediato.

Asociaciones y Asignaciones

```
let x = 10;  
let y = x;
```



```
let x = String::from("hola");  
let y = x;
```



En Rust, las "asociaciones" tras una declaración inicial y las "asignaciones" propiamente dichas a variables mutables siguen exactamente la misma semántica de copia o movimiento).

Paso de Parámetros

```
fn foo(x: i32) {  
    // ...  
}  
  
fn main() {  
    let x = 27;  
    foo(x);  
    println!("{x}");  
}
```

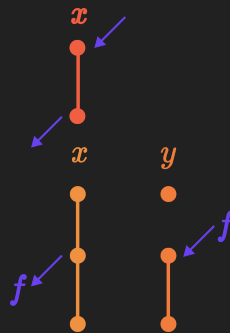


```
fn foo(x: String) {  
    // ...  
}  
  
fn main() {  
    let x = String::from("hola");  
    foo(x);  
    println!("{x}");  
}
```

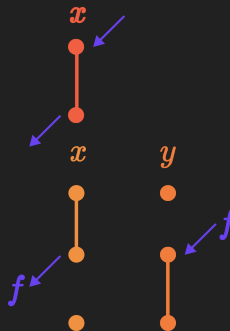


Retornos

```
fn foo(x: i32) {  
    x  
}  
  
fn main() {  
    let x = 27;  
    let y = foo(x);  
    println!("{x}");  
}
```

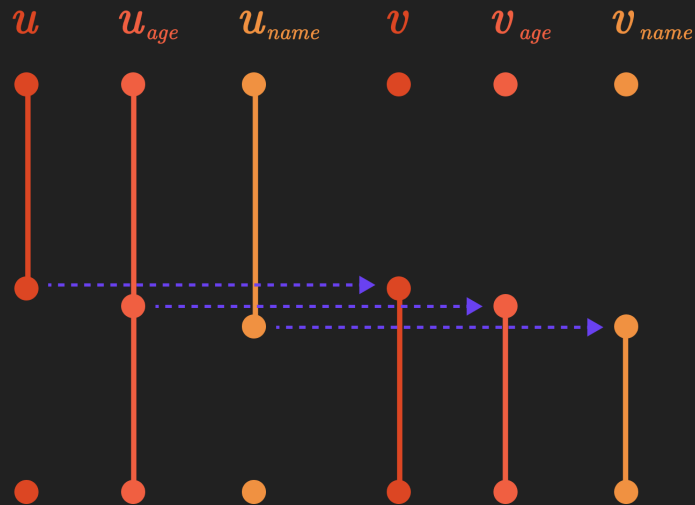


```
fn foo(x: String) {  
    x  
}  
  
fn main() {  
    let x = String::from("hola");  
    let y = foo(x);  
    println!("{y}");  
}
```



Structs

```
struct User {  
    name: String,  
    age: u8,  
}  
  
fn main() {  
    let u = User {  
        name: String::from("Giovanny"),  
        age: 25,  
    };  
  
    let v = u;  
}
```



El Costo del Ownership Estricto

El sistema de *ownership* garantiza seguridad, pero introduce una limitación práctica: **pasar datos del Heap a una función los consume**. Devolver el valor en cada función para recuperarlo es tedioso y no escala. Rust resuelve esto con un mecanismo complementario: el **Borrowing**.

Borrowing y Lifetimes

1. Para flexibilizar el sistema de *ownership*, Rust implementa un mecanismo complementario llamado ***borrow checker***. Este sistema establece reglas sobre cómo tu programa puede acceder a los datos, especialmente cuando múltiples partes necesitan leer o modificar la misma información.
2. Esto previene errores complejos (*bugs*) cuando diferentes secciones del programa intentan manipular los mismos datos de forma simultánea.

Referencias

En Rust, una **referencia** (`&`) es un puntero que permite acceder a un dato sin tomar su propiedad (*ownership*).

- **Préstamo:** El dueño original conserva el control del dato; solo le da un permiso temporal a otra variable.
- **Sintaxis:** Se utiliza el operador `&` para crear una referencia de lectura, y `&mut` para una referencia que permita modificar el dato.

Reglas del borrow checker

Para asegurar que estos préstamos temporales no corrompan la memoria ni causen problemas al compilar, el *borrow checker* vigila estrictamente dos condiciones:

1. Para un instante de tiempo dado, está permitida una y sólo una referencia mutable ó cualquier cantidad de referencias inmutables.
2. Toda referencia debe ser siempre válida.

Válido: Muchas lecturas

```
fn main() {  
    let x = String::from("hola");  
    let r1 = &x;  
    let r2 = &x;  
    println!("{r1} y {r2}");  
}
```

Error: Mezcla de lectura y escritura

```
fn main() {  
    let mut x = String::from("hola");  
    let r1 = &x;  
    let r2 = &mut x;  
    println!("{r1}");  
}
```

Puedes tener infinitas lecturas, pero si creas una referencia mutable (`&mut`), no puede existir nadie más accediendo a ese dato al mismo tiempo.

Tipos de lifetimes

Lifetimes concretos L

- Son tiempos de vida reales y físicos de las variables en memoria.
- Están determinados por el lugar donde se crean las variables y los brackets que marcan su destrucción.
- Son estáticos y ocurren al ejecutar el bloque de código.

Lifetimes genéricos $'a$

- Son variables abstractas que funcionan como parámetros de tiempo.
- No representan un tiempo fijo, sino una relación o un "cupó".

1. El *borrow checker* evalúa constantemente que los **lifetimes concretos** (L) de las referencias se ajusten de forma segura a los límites marcados por los **lifetimes genéricos** ($'a$).
2. En términos simples: vigila estrictamente que ninguna referencia viva más tiempo que el dueño original de los datos, evitando así punteros colgados (*dangling pointers*).

¿Por qué se necesitan los lifetimes genéricos?

1. **Relación de supervivencia:** Son necesarios para indicarle al compilador qué vínculo tienen las referencias entre sí cuando no puede deducirlo por su cuenta.
2. **Garantía de seguridad:** Se aplican para asegurar que un dato retornado (o guardado en una `struct`) nunca viva más tiempo que la referencia original que le dio origen.

Ecuación general de los lifetimes

Sean L y $'a$ lifetimes, s y t referencias de entrada y r la referencia de salida en una función. El borrow checker evalúa que:

$$\max(L(r)) \leq 'a \mid 'a = \min(L(s), L(t))$$

para determinar que toda referencia debe ser siempre válida durante el flujo del programa.

Evaluación de las referencias

```
fn foo<'a>(a: &'a str, b: &'a str) → &'a str {  
    if a.len() ≥ b.len() {a} else {b}  
}  
  
fn main() {  
    let s = String::from("Hello world in english!");  
    let t = String::from("Hola mundo en español!");  
    let r = foo(&s, &t);  
    println!("{}", r);  
}
```

$$'a = \min(L(s), L(t))$$

Sustituyendo los tiempos de vida reales:

$$L(s) = L(t) = L(r)$$

$$'a = L(s)$$

Condición del *borrow checker*:

$$\max(L(r)) \leq 'a$$

$$L(r) \leq L(s) \Rightarrow \text{Válido}$$

Seguridad de Memoria en Rust

Al trabajar en conjunto, el sistema de **Ownership**, el **Borrow Checker** y las reglas de **Lifetimes** eliminan los errores de gestión de memoria más comunes de la industria:

- **Punteros nulos:** Rust garantiza que toda referencia apunte siempre a un lugar válido en memoria.
- **Desbordamientos de búfer:** Valida estrictamente que las lecturas y escrituras ocurran dentro de los límites de los arreglos.
- **Punteros colgados / Doble liberación:** Erradicados por completo gracias al control estricto de ámbitos de vida.

- **Carreras de datos (Data races):** Impide el acceso simultáneo y desordenado a un mismo dato en entornos multihilo.
- **Comportamientos indefinidos:** El compilador rechaza cualquier zona gris de la memoria (*undefined behavior*).
- **Cero Garbage Collector:** No se necesita pausar el programa en ejecución; todo se resuelve al compilar.

Conclusión: Rust traslada la seguridad de la memoria al **tiempo de compilación**. Si tu código compila, es matemáticamente seguro en memoria.

Enums, Option y Result

Enums

¿Qué es un enum?

1. Un **struct** agrupa varios valores que existen **todos a la vez** (un `Punto` siempre tiene `x` y `y`).
2. Un **enum** representa un valor que puede ser **una entre varias variantes posibles** — nunca dos al mismo tiempo.

```
enum Direccion {  
    Norte,  
    Sur,  
    Este,  
    Oeste,  
}  
  
fn main() {  
    let rumbo = Direccion::Norte;  
}
```

Variantes con Datos

Cada variante puede llevar sus propios datos asociados, como si fuera un mini-struct:

```
enum Forma {  
    Circulo(f64),           // radio  
    Rectangulo(f64, f64), // ancho, alto  
}  
  
fn area(f: &Forma) -> f64 {  
    match f {  
        Forma::Circulo(r) => std::f64::consts::PI * r * r,  
        Forma::Rectangulo(a, h) => a * h,  
    }  
}
```

- `match` (ya visto en Estructuras de Control) obliga a manejar cada variante — si falta una, el código no compila.

Option

El Problema: Ausencia de Valor

Muchos lenguajes usan `null` para representar "no hay valor". Rust no tiene `null` — el mismo problema se resuelve con un enum de la librería estándar:

```
enum Option<T> {  
    Some(T),  
    None,  
}
```

`Option<T>` no es un caso especial del lenguaje — es un enum como cualquier otro, solo que tan común que el compilador lo trae disponible sin necesidad de importarlo.

Usar Option

```
fn buscar(nums: &[i32], objetivo: i32) → Option<usize> {  
    for (i, &n) in nums.iter().enumerate() {  
        if n == objetivo {  
            return Some(i);  
        }  
    }  
    None  
}  
  
fn main() {  
    match buscar(&[3, 7, 1], 7) {  
        Some(i) ⇒ println!("encontrado en {i}"),  
        None ⇒ println!("no encontrado"),  
    }  
}
```

- `Some(i)` envuelve el valor cuando existe; `None` marca su ausencia.
- El compilador obliga a manejar ambos casos — no hay forma de "olvidarse" de comprobar si hay valor, a diferencia de un `null` que puede fallar en tiempo de ejecución.

`if let` y `while let` sobre `Option`

Cuando solo interesa un caso (usualmente `Some`), `match` es más verboso de lo necesario — para eso ya viste `if let` y `while let`:

```
let dato: Option<i32> = Some(5);

if let Some(v) = dato {
    println!("hay un valor: {v}");
}
```

```
let mut pila: Vec<Option<i32>> =
    vec![Some(1), Some(2), None];

while let Some(v) = pila.pop() {
    println!("{v:?}");
}
```

Combinators Comunes

- `.unwrap()`: extrae el valor de `Some`, o hace *panic* si es `None` — solo conviene cuando la presencia del valor está garantizada.
- `.is_some()` / `.is_none()`: preguntan por el caso sin extraer el valor.
- `.map(f)`: transforma el valor interno si existe, sin tener que escribir un `match` manualmente.
- `.take()`: extrae el valor y deja `None` en su lugar.

```
let a: Option<i32> = Some(3);

let b = a.map(|v| v * 2); // Some(6)
let c = a.unwrap_or(0);  // 3 (o 0 si a fuera None)
```

Un Patrón que Reaparece

El patrón `Option<Box<Nodo>>` de la siguiente sección (Listas Enlazadas, Árboles) es exactamente esto: un enum con dos variantes, `Some` y `None`, sobre el que se hace *pattern matching* para decidir si seguir recorriendo la estructura o detenerse — y `Option::take()` es el mismo combinator mencionado aquí.

Result

El Problema: Operaciones que Fallan

1. `Option` modela la **ausencia** de un valor. Pero muchas operaciones no solo pueden "no dar nada": pueden **fallar por una razón concreta** (archivo inexistente, texto que no es un número, red caída).
2. En otros lenguajes eso se maneja con excepciones. Rust no tiene excepciones: el error viaja **en el tipo de retorno**, igual que `Option`.

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

`Result<T, E>` es otro enum de la librería estándar: `Ok(valor)` si salió bien, `Err(error)` si falló. Al ser parte del tipo, el compilador no permite ignorar la posibilidad de error.

Option vs. Result

```
// ¿hay valor o no?  
fn buscar(v: &[i32]) → Option<usize>  
  
// Some(3) / None
```

```
// ¿salió bien o por qué falló?  
fn leer(t: &str) → Result<i32, ParseIntError>  
  
// Ok(42) / Err(...)
```

Regla práctica: cuando la ausencia es **normal y esperada** (el elemento no está), corresponde **Option**. Cuando es un **fallo que amerita explicación**, corresponde **Result** — el **E** carga el motivo.

Manejar un Result

```
fn main() {  
    let texto = "42";  
  
    match texto.parse::<i32>() {  
        Ok(n)    => println!("el número es {n}"),  
        Err(e)   => println!("no se pudo convertir: {e}"),  
    }  
}
```

- `match` obliga a cubrir ambos casos, igual que con `Option`.
- `if let Ok(n) = texto.parse::<i32>()` sirve cuando solo interesa el caso exitoso.
- `.unwrap()` extrae el `Ok` y hace *panic* si es `Err`; `.expect("mensaje")` es igual pero con un mensaje propio.

El Operador `?`

Escribir un `match` por cada operación que puede fallar es insostenible. El operador `?` lo resume: **si es `Ok`, extrae el valor; si es `Err`, retorna el error de inmediato.**

```
// con match: verboso
fn sumar(a: &str, b: &str)
    → Result<i32, ParseIntError> {
    let x = match a.parse::<i32>() {
        Ok(n) ⇒ n,
        Err(e) ⇒ return Err(e),
    };
    let y = match b.parse::<i32>() {
        Ok(n) ⇒ n,
        Err(e) ⇒ return Err(e),
    };
    Ok(x + y)
}
```

```
// con ?: la misma lógica
fn sumar(a: &str, b: &str)
    → Result<i32, ParseIntError> {
    let x = a.parse::<i32>()?;
    let y = b.parse::<i32>()?;
    Ok(x + y)
}
```

`?` solo puede aparecer dentro de una función que **devuelva `Result`** (o **`Option`**): requiere un destino al que propagar el error. Por eso el retorno sigue siendo **`Result`** y el éxito se envuelve en **`Ok(...)`**.

Result Estaba a la Vista

Cada `.unwrap()` aplicado sobre algo que no era un `Option` operaba en realidad sobre un `Result`:

```
let re = Regex::new(r"[0-9]+").unwrap(); // Result<Regex, Error>
let n: i32 = "42".parse().unwrap();      // Result<i32, ParseIntError>
```

`.unwrap()` es admisible en un *trial* o un prototipo, donde un *panic* no tiene consecuencias. En código real se prefiere `?` para propagar el error, o `match` para decidir qué hacer con él.

Y el patrón se repite: `Option` y `Result` son **enums con dos variantes** sobre los que se hace *pattern matching* — la misma "cebolla" que se pela con `match`, `if let` o `?`.

Estructuras de Datos

Vec

¿Qué es un Vec?

1. Un `Vec<T>` es una **secuencia homogénea y de tamaño variable** — todos sus elementos son del mismo tipo `T`, pero puede crecer y encogerse en tiempo de ejecución.
2. A diferencia de un arreglo `[T; N]` (tamaño fijo, conocido en tiempo de compilación), un `Vec` no necesita saber cuántos elementos tendrá de antemano.

```
let arreglo: [i32; 3] = [1, 2, 3]; // tamaño fijo: siempre 3
let mut lista: Vec<i32> = Vec::new(); // vacío, crecerá
lista.push(1);
lista.push(2);
```

Dónde Vive un Vec

El contraste con Tipos de Memoria: un `Vec` reparte su información entre stack y heap.

- En el **stack** vive el `Vec` en sí: tres datos de tamaño fijo — un **puntero** al buffer, la **longitud** (`len`) y la **capacidad** (`cap`).
- En el **heap** viven los **elementos** reales, en un buffer contiguo al que apunta ese puntero.

Por eso un `Vec` sigue las reglas de *ownership* de los datos en heap: asignarlo o pasarlo a una función lo **mueve** (no se copia el buffer completo).

Construcción y Operaciones

```
let mut v = Vec::new();    // vacío
let mut w = vec![10, 20];  // macro: con valores iniciales

v.push(1);                 // agrega al final
v.push(2);
let ultimo = v.pop();      // quita el último y lo devuelve

let n = w.len();           // cantidad de elementos
```

`.pop()` no devuelve un `i32` directo, sino un `Option<T>`: `Some(valor)` si había elementos, `None` si el `Vec` estaba vacío. Es el mismo `Option` de la sección anterior — el compilador obliga a considerar el caso vacío.

Indexado Seguro vs. Directo

Hay dos formas de acceder a un elemento, y difieren en qué pasa si el índice no existe:

```
let v = vec![10, 20, 30];

// acceso directo: panic si
// el índice no existe
let x = v[1]; // 20
let y = v[9]; // panic!
```

```
let v = vec![10, 20, 30];

// acceso seguro: devuelve Option
let x = v.get(1); // Some(&20)
let y = v.get(9); // None
```

`.get(i)` devuelve `Option<&T>` en vez de arriesgar un *panic* — de nuevo, `Option` modelando la posible ausencia.

Slices: una Vista Prestada

Un **slice** `&[T]` es una referencia a una porción contigua de un `Vec` (o arreglo), sin tomar su propiedad — es *borrowing* aplicado a colecciones.

```
fn suma(nums: &[i32]) -> i32 {
    let mut total = 0;
    for &n in nums {
        total += n;
    }
    total
}

fn main() {
    let v = vec![1, 2, 3, 4];
    let total = suma(&v);           // &Vec<i32> se presta como &[i32]
    let parcial = suma(&v[1..3]); // solo una parte
}
```

- Recibir `&[T]` en vez de `&Vec<T>` hace la función más general: acepta tanto `Vec` como arreglos.
- El slice es un préstamo: `v` sigue siendo dueño de sus datos.

Iteradores

Tres Formas de Iterar

La forma de recorrer un `Vec` decide qué le pasa a sus elementos — y es exactamente la distinción de *ownership* y *borrowing* ya vista:

- `.iter()` — presta cada elemento como `&T` (solo lectura). El `Vec` sobrevive intacto.
- `.iter_mut()` — presta cada elemento como `&mut T` (permite modificarlo en su lugar).
- `.into_iter()` — **consume** el `Vec`: entrega cada elemento por valor (`T`) y el `Vec` deja de existir.

```
let v = vec![1, 2, 3];

for x in v.iter() {      // x: &i32
    println!("{x}");
}

// v sigue disponible aquí
```

Adaptadores Perezosos

Un **adaptador** transforma un iterador en otro, pero **no ejecuta nada todavía** — solo describe qué hacer.

```
let v = vec![1, 2, 3, 4];  
  
let pares = v.iter().filter(|&n| n % 2 == 0);  
let doubles = v.iter().map(|n| n * 2);
```

- `.filter(pred)` — deja pasar solo los elementos que cumplen el predicado.
- `.map(f)` — transforma cada elemento aplicando la función `f`.

Estas líneas no recorren nada aún. Un iterador es **perezoso**: describe una receta que solo se ejecuta cuando un **consumidor** la pide.

Consumidores

Un **consumidor** recorre el iterador de verdad y produce un resultado final:

```
let v = vec![1, 2, 3, 4];

let total: i32 = v.iter().sum();           // 10
let cuantos = v.iter().count();           // 4

let dobles: Vec<i32> = v.iter()
    .map(|n| n * 2)
    .collect();                           // [2, 4, 6, 8]
```

- `.sum()` / `.count()` — reducen el iterador a un solo valor.
- `.collect()` — junta todos los elementos en una colección (aquí, un `Vec` nuevo).

Encadenar la Receta

El patrón real: encadenar adaptadores perezosos y cerrar con un consumidor. Nada se ejecuta hasta el `.collect()` final.

```
let v = vec![1, 2, 3, 4, 5, 6];

let resultado: Vec<i32> = v.iter()
    .filter(|&&n| n % 2 == 0) // solo pares: 2, 4, 6
    .map(|n| n * 10)         // por diez: 20, 40, 60
    .collect();             // junta en Vec: [20, 40, 60]
```

Cuando el compilador no puede deducir el tipo de destino de `.collect()`, se anota con *turbofish*: `.collect::<Vec<i32>>()`. Es la misma información que el `: Vec<i32>` de la variable, escrita del lado del método.

HashMap y HashSet

Colecciones asociativas

HashMap — clave → valor

1. Un `HashMap<K, V>` asocia cada **clave** con un **valor**, y accede por clave en **O(1) promedio** — no recorre la colección.
2. No garantiza ningún orden de los elementos.

```
use std::collections::HashMap;

let mut edades: HashMap<String, u32> = HashMap::new();
edades.insert("Ana".to_string(), 30);

if let Some(e) = edades.get("Ana") {    // get → Option<u32>
    println!("{e}");
}
```

`get` devuelve `Option<V>`, no el valor directo — de nuevo `Option` modelando que **la clave podría no existir**.

Contar con `entry`

El patrón más común con `HashMap`: acumular o contar. `entry(k).or_insert(v)` devuelve una **referencia mutable** al valor, insertándolo primero si la clave no existía.

```
let mut freq: HashMap<char, i32> = HashMap::new();

for c in "banana".chars() {
    *freq.entry(c).or_insert(0) += 1;
}
// {'b': 1, 'a': 3, 'n': 2}
```

`entry` evita el doble trabajo de "¿ya existe la clave? si no, insértala; luego increméntala" en una sola operación, sin dos búsquedas separadas.

HashSet — pertenencia sin duplicados

1. Un `HashSet<T>` guarda elementos **únicos**; responde *¿está o no está?* en **$O(1)$ promedio**.
2. Es un `HashMap` donde solo importa la clave (el valor es `()`).

```
use std::collections::HashSet;

let mut vistos: HashSet<i32> = HashSet::new();

for n in [1, 2, 2, 3] {
    if !vistos.insert(n) {           // insert → bool
        println!("repetido: {n}"); // 2
    }
}
```

`insert` devuelve `false` si el elemento **ya estaba** → detecta duplicados en una sola operación, sin un `contains` aparte.

Structs

¿Qué es un struct?

1. Un **struct** agrupa varios valores relacionados bajo un solo tipo, cada uno con su propio nombre (*campo*) y tipo.
2. A diferencia de una tupla, cada campo tiene un nombre — el código ya no depende del orden para saber qué representa cada valor.

```
struct Punto {  
    x: f64,  
    y: f64,  
}  
  
fn main() {  
    let origen = Punto { x: 0.0, y: 0.0 };  
    println!("({}, {})", origen.x, origen.y);  
}
```

impl : Funciones Asociadas y Métodos

```
struct Punto {  
    x: f64,  
    y: f64,  
}  
  
impl Punto {  
    // funcion asociada: no recibe self,  
    // se invoca como Punto::origen()  
    fn origen() -> Self {  
        Self { x: 0.0, y: 0.0 }  
    }  
  
    // metodo: recibe &self, se invoca  
    // sobre una instancia ya creada  
    fn distancia(&self, otro: &Punto) -> f64 {  
        ((self.x - otro.x).powi(2)  
         + (self.y - otro.y).powi(2)).sqrt()  
    }  
}
```

- `Self` (mayúscula) es un alias del tipo que implementa el bloque — evita repetir `Punto` dentro del `impl`.
- `&self` presta la instancia de forma inmutable; `&mut self` la presta de forma mutable si el método necesita modificar campos.
- Las funciones asociadas (sin `self`) suelen usarse como constructores, por convención llamadas `new`.

Un Método es Azúcar Sintáctico sobre un Préstamo

Cuando se llama a un método con `caja.metodo(...)`, Rust no hace nada mágico: por debajo, toma el préstamo que `self` pide y lo pasa como primer argumento a la función asociada.

```
struct Caja {  
    valor: i32,  
}  
  
impl Caja {  
    fn ver(&self) -> &i32 {  
        &self.valor  
    }  
  
    fn cambiar(&mut self, n: i32) {  
        self.valor = n;  
    }  
}
```

```
let mut caja = Caja { valor: 5 };  
  
caja.ver();  
// = Caja::ver(&caja)  
  
caja.cambiar(10);  
// = Caja::cambiar(&mut caja, 10)
```

`&self` y `&mut self` no son un mecanismo aparte del *borrowing* — son el mismo `&`/`&mut` de siempre, tomado automáticamente sobre la instancia. Por eso las reglas del borrow checker aplican igual a las llamadas de métodos.

Las Reglas de Borrowing También Aplican a Métodos

Como `caja.ver()` es `Caja::ver(&caja)` y `caja.cambiar(10)` es `Caja::cambiar(&mut caja, 10)`, la regla ya vista — *una mutable o varias inmutables, nunca ambas al mismo tiempo* — se cumple igual entre llamadas a métodos:

Válido: dos préstamos de solo lectura

```
fn main() {  
    let caja = Caja { valor: 5 };  
    let a = caja.ver();  
    let b = caja.ver();  
    println!("{a} y {b}");  
}
```

Error: préstamo vivo + método mutable

```
fn main() {  
    let mut caja = Caja { valor: 5 };  
    let v = caja.ver();    // &self activo  
    caja.cambiar(10);      // &mut self: conflicto  
    println!("{v}");  
}
```

`caja.ver()` solo lee — se pueden llamar tantas veces como se quiera. Pero mientras su resultado siga en uso, `caja.cambiar(10)` pide un `&mut self` que no puede coexistir con ese préstamo de lectura.

Derive Traits

Escribir a mano comparaciones, formateo o clonado para cada struct sería repetitivo. El atributo `#[derive(...)]` le pide al compilador que genere esa implementación automáticamente.

```
#[derive(Debug, Clone, PartialEq, Eq)]
struct Punto {
    x: i32,
    y: i32,
}
```

- **Debug**: habilita el formato `{:?}` para imprimir el struct completo (útil para depurar).
- **Clone**: habilita `.clone()` para duplicar el valor explícitamente, incluso si vive en el heap.
- **PartialEq** / **Eq**: habilitan comparar dos instancias con `==`, campo por campo.

Unit Structs vs. Structs con Campos

1. No todos los structs necesitan guardar datos. Un **unit struct** se declara sin campos ni llaves, solo como marcador de tipo:

```
struct Marcador;  
  
fn main() {  
    let m = Marcador;  
}
```

2. Sirven para representar un concepto o agrupar funciones relacionadas bajo un `impl`, sin necesitar una instancia con datos — equivalen a un espacio de nombres con identidad de tipo propia.
3. Un struct con campos, en cambio, existe para *cargar información*: cada instancia es distinta porque sus valores lo son.

Regla práctica: si el tipo solo necesita comportamiento (métodos) sin estado propio, un unit struct basta; si necesita representar datos distintos por instancia, corresponde declarar campos.

Listas Enlazadas

El problema: tipos recursivos

1. Una lista enlazada es una secuencia de nodos donde cada nodo apunta al siguiente. La forma más directa de modelarlo sería:

```
struct Nodo {  
    valor: i32,  
    siguiente: Nodo,  
}
```

2. Este código no compila. Rust necesita conocer el tamaño exacto de cada tipo en tiempo de compilación, pero `Nodo` contiene otro `Nodo`, que contiene otro `Nodo` ... un tamaño infinito.

Box<T> : Indirección al Heap

Box<T> es un puntero inteligente que guarda un valor en el heap y deja en el stack solo un puntero de tamaño fijo hacia él.

```
struct Nodo {  
    valor: i32,  
    siguiente: Box<Nodo>,  
}
```

Al envolver el campo recursivo en un **Box**, el compilador ya no necesita conocer el tamaño de **Nodo** completo — solo el tamaño de un puntero.

Pero sigue faltando algo: ¿cómo se representa el final de la lista, donde no hay "siguiente" nodo?

`Option<Box<Nodo>>` : Representar el Final

La lista termina cuando un nodo no tiene siguiente. Esa ausencia se modela con `Option`, no con un valor especial como `null` (que Rust no tiene):

```
struct Nodo {  
    valor: i32,  
    siguiente: Option<Box<Nodo>>,  
}  
  
fn main() {  
    let ultimo = Nodo { valor: 3, siguiente: None };  
    let segundo = Nodo { valor: 2, siguiente: Some(Box::new(ultimo)) };  
    let primero = Nodo { valor: 1, siguiente: Some(Box::new(segundo)) };  
}
```

- `None` marca el final de la lista.
- `Some(Box::new(nodo))` envuelve el siguiente nodo, ya en el heap.

Recorrer sin robar la propiedad

Para leer la lista sin consumirla, se recorre con una **referencia** que avanza nodo a nodo — el mismo patrón de *borrowing* visto en Borrowing y Lifetimes:

```
fn imprimir(nodo: &Option<Box<Nodo>>) {  
    let mut actual = nodo;  
    while let Some(n) = actual {  
        println!("{}", n.valor);  
        actual = &n.siguiete;  
    }  
}
```

- `actual` es una referencia (`&Option<Box<Nodo>>`), no el dueño de los nodos.
- En cada vuelta, `actual` avanza al campo `siguiete` del nodo actual — nunca se mueve el nodo, solo el "dedo" que lo señala.

Mover en vez de prestar: `Option::take()`

Cuando sí se necesita tomar posesión de un nodo (por ejemplo, para invertir la lista), prestar no basta — hay que mover el valor y dejar algo válido en su lugar.

```
fn quitar_primer_nodo(lista: &mut Option<Box<Nodo>>) -> Option<Box<Nodo>> {  
    lista.take()  
}
```

`Option::take()` extrae el valor de una `Option`, dejando `None` en su lugar. Así se puede mover un nodo sin dejar el campo original en un estado inválido.

- Antes de `take()`: `lista` posee el nodo.
- Después de `take()`: `lista` queda en `None`, y el nodo pasa a quien recibió el valor devuelto.

Árboles Binarios

De listas a árboles

1. Una lista enlazada es estrictamente lineal: cada nodo tiene un único siguiente. Un **árbol binario** relaja esa restricción — cada nodo puede tener hasta dos hijos.

```
struct Nodo {  
    valor: i32,  
    izquierda: Option<Box<Nodo>>,  
    derecha: Option<Box<Nodo>>,  
}
```

2. Con `Box`, cada nodo sigue siendo dueño exclusivo de sus hijos, igual que en la lista enlazada. Esto funciona mientras el árbol se recorra y modifique desde un único punto de entrada.

Cuando un dueño no basta

Algunos escenarios rompen el supuesto de "un solo dueño":

- Compartir el mismo nodo desde **más de un lugar** (por ejemplo, una estructura que reorganiza o balancea sus propios subárboles).
- Necesitar **mutar** un nodo a través de una referencia compartida, algo que el borrow checker prohíbe en tiempo de compilación con referencias normales.

`Box<T>` resuelve "¿quién es dueño?" con una respuesta única. Cuando la respuesta necesita ser "varios, y a veces hay que modificarlo", Rust ofrece `Rc<RefCell<T>>`.

`Rc<T>` : Conteo de Referencias

`Rc<T>` (*Reference Counted*) permite que varios dueños compartan el mismo valor en el heap, llevando la cuenta de cuántas referencias existen.

```
use std::rc::Rc;

let a = Rc::new(5);
let b = Rc::clone(&a); // no copia el 5, incrementa el contador

println!("referencias: {}", Rc::strong_count(&a)); // 2
```

- `.clone()` sobre un `Rc` es barato: solo incrementa un contador, no duplica el dato.
- El valor se libera únicamente cuando el contador llega a cero (el último dueño sale de scope).
- `Rc<T>` por sí solo **no permite mutar** el valor compartido — todas las referencias son de solo lectura.

RefCell<T> : Mutabilidad en Tiempo de Ejecución

`RefCell<T>` mueve la verificación de las reglas de *borrowing* (una referencia mutable o varias inmutables) del tiempo de compilación al tiempo de ejecución.

```
use std::cell::RefCell;

let celda = RefCell::new(5);

*celda.borrow_mut() += 1;
println!("{}", celda.borrow()); // 6
```

Si en tiempo de ejecución se viola la regla (dos préstamos mutables simultáneos, por ejemplo), el programa hace *panic* en vez de no compilar.

`Rc<RefCell<T>>` : Compartir y Mutar

Combinando ambos, un árbol puede tener nodos con **múltiples dueños** que además pueden **mutarse**:

```
use std::cell::RefCell;
use std::rc::Rc;

struct Nodo {
    valor: i32,
    izquierda: Option<Rc<RefCell<Nodo>>>,
    derecha: Option<Rc<RefCell<Nodo>>>,
}
```

- `Rc` responde "¿quién puede acceder a este nodo?" → varios.
- `RefCell` responde "¿puede alguien modificarlo?" → sí, verificado en ejecución.

Recorridos: Preorden, Inorden, Postorden

Los tres recorridos clásicos visitan los mismos tres elementos (nodo, izquierda, derecha), solo cambia el orden:

Preorden

```
fn preorden(n: &Nodo) {  
    visitar(n);  
    if let Some(i) = &n.izquierda {  
        preorden(i);  
    }  
    if let Some(d) = &n.derecha {  
        preorden(d);  
    }  
}
```

nodo → izq → der

Inorden

```
fn inorden(n: &Nodo) {  
    if let Some(i) = &n.izquierda {  
        inorden(i);  
    }  
    visitar(n);  
    if let Some(d) = &n.derecha {  
        inorden(d);  
    }  
}
```

izq → nodo → der

Postorden

```
fn postorden(n: &Nodo) {  
    if let Some(i) = &n.izquierda {  
        postorden(i);  
    }  
    if let Some(d) = &n.derecha {  
        postorden(d);  
    }  
    visitar(n);  
}
```

izq → der → nodo

Cierre: ¿Cuál Indirección Usar?

- `Box<T>` — un único dueño, estructura estrictamente jerárquica (listas, árboles simples sin reestructuración).
- `Rc<T>` — múltiples dueños de solo lectura.
- `Rc<RefCell<T>>` — múltiples dueños que además necesitan mutar el valor compartido.

La elección no es estética: refleja cuántos dueños tiene realmente el dato y si necesita cambiar después de compartirse.

Big-O de un vistazo

Estructura	Acceso	Búsqueda	Inserción	Borrado
Vec / array (índice)	$O(1)$	$O(n)$	$O(1)^*$ al final	$O(n)$
HashMap / HashSet	$O(1)^\dagger$	$O(1)^\dagger$	$O(1)^\dagger$	$O(1)^\dagger$
Linked List (Box)	$O(n)$	$O(n)$	$O(1)$ cabeza	$O(1)$ cabeza
BST balanceado	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Heap (BinaryHeap)	$O(1)$ min/max	$O(n)$	$O(\log n)$	$O(\log n)$ pop

* `push` / `pop` al final: $O(1)$ amortizado.

† caso promedio; peor caso $O(n)$ por colisiones.

De diario: Vec, HashMap, HashSet.

De aprendizaje: listas y árboles importan sobre todo por lo que enseñan del ownership (`Box`, `Rc`, `RefCell`), más que por lo seguido que los uses.

Traits y Generics

Traits

¿Qué es un trait?

1. Un **trait** define un **contrato de comportamiento**: un conjunto de métodos que un tipo promete implementar.
2. No dice *qué* es el tipo, sino *qué sabe hacer*. Varios tipos distintos pueden cumplir el mismo trait.

```
trait Descriptible {  
    fn describir(&self) → String;  
}  
  
struct Punto { x: f64, y: f64 }  
struct Usuario { nombre: String }  
  
impl Descriptible for Punto {  
    fn describir(&self) → String { format!("({}, {})", self.x, self.y) }  
}  
  
impl Descriptible for Usuario {  
    fn describir(&self) → String { format!("usuario {}", self.nombre) }  
}
```

Métodos por Default

Un trait puede traer implementaciones ya hechas. El tipo que lo implementa puede **usarlas tal cual o sobrescribirlas**.

```
trait Descriptible {  
    fn describir(&self) → String;  
  
    // default: se apoya en describir()  
    fn presentarse(&self) → String {  
        format!("Soy {}", self.describir())  
    }  
}
```

Así se evita repetir la misma lógica en cada implementación: el trait define lo mínimo obligatorio (**describir**) y deriva el resto (**presentarse**) a partir de ello.

derive es Auto-Implementar Traits

Los traits ya aparecían sin nombrarse: `#[derive(...)]` le pide al compilador que **genere la implementación automáticamente**.

DERIVE	QUÉ HABILITA
Debug	imprimir con <code>{:?}</code>
Clone	el método <code>.clone()</code>
PartialEq / Eq	comparar con <code>==</code>
PartialOrd / Ord	<code><</code> , <code>></code> , y ordenar
Hash	ser llave de un <code>HashMap</code> / <code>HashSet</code>

De ahí que un tipo requiera `Hash + Eq` para ser llave de un `HashMap`, y `Ord` para usarse en `sort()` o en un `BinaryHeap`: esas estructuras **exigen** esos traits.

Traits Presentes en las Secciones Anteriores

CONSTRUCCIÓN USADA

```
for x in v.iter() · .map() · .filter()
```

```
println!("{:?}", v)
```

```
v.sort()
```

closures como `|x| x * 2`

```
String::from("x")
```

EL TRAIT DETRÁS

`Iterator` — cada adaptador vive en este trait

`Debug`

`Ord` sobre los elementos

`Fn` / `FnMut` / `FnOnce`

`From`

Los closures también son traits: `Fn` (solo lee lo que captura), `FnMut` (lo muta) y `FnOnce` (lo consume). De ahí que `.map()` acepte cualquier closure: su firma pide un tipo que cumpla uno de esos traits, no un tipo concreto.

Generics

El Problema: Repetir por Cada Tipo

Sin genéricos habría que escribir la misma función una vez por tipo:

```
fn mayor_i32(a: i32, b: i32) → i32 { if a > b { a } else { b } }  
fn mayor_f64(a: f64, b: f64) → f64 { if a > b { a } else { b } }
```

Un **genérico** abstrae el tipo con un parámetro (**T**), y el compilador genera la versión concreta para cada uso:

```
fn mayor<T: PartialOrd>(a: T, b: T) → T {  
    if a > b { a } else { b }  
}
```

Trait Bounds: Donde se Juntan Traits y Generics

`<T: PartialOrd>` es un **trait bound**: no acepta *cualquier* tipo, sino cualquiera **que cumpla ese trait**.

Se lee: "**T** es *cualquier tipo*, **siempre que se pueda comparar con** **>**". Sin el bound el compilador rechaza `a > b`, porque no todo tipo sabe compararse.

```
// un bound
fn mayor<T: PartialOrd>(a: T, b: T) -> T

// varios bounds
fn mostrar<T: Debug + Clone>(x: T)
```

```
// sintaxis `where`: equivalente, y
// más legible con muchos bounds
fn procesar<T, U>(a: T, b: U)
where
    T: Debug + Clone,
    U: PartialOrd,
{ }
```

Structs Genéricos

El parámetro de tipo también sirve para definir estructuras que funcionan con cualquier contenido:

```
struct Par<T> {  
    primero: T,  
    segundo: T,  
}  
  
impl<T: PartialOrd> Par<T> {  
    fn mayor(&self) → &T {  
        if self.primero > self.segundo { &self.primero } else { &self.segundo }  
    }  
}
```

- `struct Par<T>` declara el parámetro; `impl<T: ...> Par<T>` lo vuelve a declarar para el bloque de métodos.
- El bound puede vivir solo en el `impl`: así el struct admite cualquier `T`, pero `mayor()` existe únicamente si `T` es comparable.

Genéricos en las Secciones Anteriores

Todas estas estructuras, ya vistas, son genéricas:

TIPO	PARÁMETROS	BOUND QUE EXIGE
<code>Vec<T></code>	el tipo de los elementos	ninguno
<code>Option<T></code> / <code>Result<T, E></code>	el valor y el error	ninguno
<code>Box<T></code> / <code>Rc<T></code> / <code>RefCell<T></code>	el dato apuntado	ninguno
<code>HashMap<K, V></code>	llave y valor	<code>K: Hash + Eq</code>
<code>BinaryHeap<T></code>	los elementos	<code>T: Ord</code>

Costo cero: el compilador genera una versión concreta de cada genérico por cada tipo usado (*monomorfización*). Un `Vec<i32>` es tan rápido como uno escrito a mano para `i32` — la abstracción no cuesta nada en tiempo de ejecución.

Apéndices

Cheatsheet

Hoja de referencia para la corrida mental

Ownership — ¿Se copia o se mueve?

Tipo de dato	Dónde vive	Al asignar / pasar / retornar	¿Sigue usable?
i32, u8, f64, bool, char	Stack	Copia (Copy)	✅ Sí
&T, &mut T	Stack	Copia el puntero	✅ Sí *
String, Vec<T>, Box<T>	Heap	Movimiento (Move)	❌ No **

Las 3 reglas del ownership

- ① Cada valor tiene un dueño (*owner*).
- ② Solo un dueño a la vez.
- ③ Cuando el dueño sale de scope, el valor se destruye (*drop*).

Gatillan transferencia: `let y = x;` · `foo(x)` · `return x;`

* según las reglas de borrowing · ** salvo `.clone()`

Borrowing y Lifetimes — ¿Es válida esta referencia?

Referencia	Permite	¿Cuántas a la vez?
<code>&T</code> (compartida)	leer	✓ muchas simultáneas
<code>&mut T</code> (exclusiva)	leer y escribir	⚠ una sola, y sin <code>&</code> activas

Regla del borrow checker: en un mismo instante puede existir **una sola** `&mut T` o bien cualquier cantidad de `&T` — pero nunca las dos a la vez. Además, toda referencia debe ser siempre válida (sin *dangling*).

Lifetimes

Ecuación: $'a = \min(L(\text{entradas}))$; válido si $\max(L(r)) \leq 'a$ — la referencia nunca vive más que su dueño.

Pelar la cebolla — leer un tipo envuelto

Envoltura	Qué representa	Cómo la abres	Qué obtienes
<code>&T</code> / <code>&mut T</code>	préstamo	<code>deref *</code> (casi siempre auto)	acceso sin mover
<code>Box<T></code>	dueño único (heap)	auto-deref *	el <code>T</code> interior
<code>Option<T></code>	¿hay valor?	<code>if let Some</code> / <code>match</code> / <code>?</code>	el <code>T</code> (o nada)
<code>Result<T,E></code>	éxito o error?	<code>match</code> / <code>?</code>	el <code>T</code> (o <code>E</code>)
<code>Rc<T></code> / <code>Arc<T></code>	propiedad compartida	auto-deref → <code>&T</code> ; <code>.clone()</code>	solo lectura; comparte dueño
<code>RefCell<T></code>	mutabilidad interior	<code>.borrow()</code> / <code>.borrow_mut()</code>	<code>Ref</code> / <code>RefMut</code> (runtime)
<code>Vec<T></code>	secuencia	<code>.iter()</code> / <code>.into_iter()</code> / <code>[i]</code>	<code>&T</code> / <code>T</code> / elemento

El método · 3 pasos

- ❶ Leer el tipo de **afuera** → **adentro**.
- ❷ Abrir cada capa con su operación y determinar qué entrega: **dueño** (move), **préstamo** (`&`) o **valor Copy**.
- ❸ Repetir hasta llegar al dato buscado.

Iteradores = el mismo patrón: `.iter()` presta, `.into_iter()` consume.

Ejemplo · el nodo del árbol

```
Option<Rc<RefCell<TreeNode>>>  
→ if let Some(rc) → &Rc<..>  
→ (Rc se desreferencia solo)  
→ .borrow() → Ref<TreeNode>  
→ .val ✓
```

Diagnóstico — el compilador se queja

Error del compilador

use of moved value

cannot borrow as mutable ... also borrowed as immutable

cannot borrow as mutable more than once

x does not live long enough

missing lifetime specifier

Qué regla violaste

usaste un valor del heap **después** de moverlo

pediste `&mut` con una `&` todavía viva

dos `&mut` al mismo tiempo

la referencia sobrevive a su dueño

retornas `&T` sin decir de cuál entrada

Fix típico

`.clone()`, o pasar `&x` en vez de `x`

termina de usar la ref immutable antes

separa los scopes de cada préstamo

el dato debe vivir al menos tanto como la ref

anota `'a` en firma y retorno

Corrida mental — tres preguntas en orden

- ① ¿El dato vive en Heap? → asignarlo o pasarlo lo **mueve** (el original queda inutilizable). ¿En Stack? → se **copia**, ambos siguen vivos.
- ② ¿Hay una `&mut` y una `&` vivas a la vez sobre el mismo dato? → prohibido: equivale a escribir mientras alguien lee.
- ③ ¿La referencia devuelta o guardada vive más que su dueño? → *dangling*, prohibido.

En corto, casi todo error de memoria es uno de tres: **un valor movido y vuelto a usar** · **lectura y escritura mezcladas** · **una referencia que vive más que su dueño**

Aplicación

El cheatsheet aplicado a los trials del repo

Aplicación · Move + `.clone()` — Reverse Linked List (206)

```
// reverse_list CONSUME la lista
pub fn reverse_list(head: Node) → Node {
    /* ... */
}

// en el test, para no perder la entrada:
let head = Solution::reverse_list(input.clone());
let ans  = Solution::get_vec(head);
```

Corrida mental

`Node = Option<Box<ListNode>>` vive en el **heap** → pasarlo a `reverse_list` lo **mueve**.

Sin `.clone()`, `input` quedaría inutilizable después de esa línea. La copia conserva el original para el resto del test.

Cheatsheet: **Heap** → **Movimiento** · ¿usable? No, salvo

`.clone()`

Aplicación · `&mut T` exclusiva — Remove Duplicates (26)

```
pub fn remove_duplicates(nums: &mut Vec<i32>) → i32 {  
    let mut set = HashSet::new();  
    nums.retain(|&x| set.insert(x));  
    nums.len() as i32  
}
```

```
let mut arr = vec![1, 1, 2];  
Solution::remove_duplicates(&mut arr);  
// arr quedó modificado in-place
```

Corrida mental

`&mut Vec<i32>` es un préstamo **exclusivo**: mientras dura, nadie más puede leer ni escribir `arr`.

Permite mutar in-place con `retain` sin mover el `Vec` ni devolver uno nuevo.

Cheatsheet: `&mut T` → una sola, y sin `&` activas

Aplicación · **&T** compartida — Longest Common Prefix (14)

```
for word in strs.iter() {    // word: &String
    if len > word.len() {
        len = word.len();
    }
}
// strs sigue disponible: se recorre
// otra vez más abajo
```

Corrida mental

.iter() entrega **&String**: préstamos **compartidos**, de los que pueden existir muchos a la vez.

strs no se mueve — sigue usable tras el bucle (de hecho se vuelve a iterar después).

Cheatsheet: **&T** → **muchas lecturas** simultáneas

Aplicación · Copy de escalares — Container With Most Water (11)

```
let mut i = 0_usize;
let mut j = height.len() - 1;

while i < j {
    let a = min(height[i], height[j]); // i32
    if height[i] < height[j] { i += 1; }
    else { j -= 1; }
}
```

Corrida mental

`i`, `j` (`usize`) y los `height[i]` (`i32`) viven en el **stack** → se **copian**.

De ahí que `i += 1` y las comparaciones funcionen libremente, sin mover ni prestar nada.

Cheatsheet: **Stack** → **Copia** · ambos siguen vivos

Aplicación · `&mut` "puntero láser" — Reverse Linked List (206)

```
let mut head: Node = None;
let mut iter: &mut Node = &mut head;

for &val in values.iter() {
    *iter = Some(Box::new(
        ListNode { val, next: None }
    ));
    iter = &mut iter.as_mut().unwrap().next;
}
head
```

Corrida mental

`iter: &mut Node` es un préstamo **exclusivo** que se va reapuntando al `next` de cada nodo.

Solo puede existir ese único `&mut` sobre la cadena a la vez — el borrow checker lo garantiza en compilación.

Cheatsheet: `&mut T` → **una sola** · [enlaza con Listas Enlazadas](#)

Aplicación · Efecto cebolla `Rc<RefCell<TreeNode>>>` — Invert Binary Tree (226)

```
type Node = Option<Rc<RefCell<TreeNode>>>>;

fn pre_order(n: &Node) {
    // n.left NO compila: n es un
    // Option, no un TreeNode
    if let Some(rc) = n {          // &Rc<..>
        let nodo = rc.borrow(); // Ref<TreeNode>
        // visitar(nodo.val);
        pre_order(&nodo.left);
        pre_order(&nodo.right);
    }
}
```

Corrida mental — 4 capas

Option ¿hay nodo? → `if let Some`

Rc propiedad compartida → deref

RefCell mutabilidad interior → `.borrow()`

TreeNode datos → ya `.left` / `.right`

No se puede mover desde atrás de un `&`: por eso `if let`

`Some(rc) = n` presta (match ergonomics), no consume.

Cheatsheet: **no sacar un valor de un préstamo** ·

`.borrow()` = lectura compartida

Aplicación · HashMap + `get` → Option — Two Sum (1)

```
let mut visto: HashMap<i32, i32> = HashMap::new();

for (idx, val) in nums.iter().enumerate() {
    let falta = target - val;
    if let Some(&i) = visto.get(&falta) {
        return vec![i, idx as i32];
    }
    visto.insert(*val, idx as i32);
}
```

Corrida mental

`visto.get(&falta)` devuelve `Option<&i32>` — el compilador obliga a manejar el caso "no está" con `if let Some`.

`Some(&i)` desestructura el `&` para copiar el `i32` (Copy), sin retener un préstamo del HashMap.

Cheatsheet: `get` → `Option` · abrir con `if let Some`

Aplicación · HashSet + insert / contains — Contains Duplicate (217)

```
let mut visto: HashSet<i32> = HashSet::new();

for n in nums.iter() {
    if visto.contains(&n) {
        return true;
    }
    visto.insert(*n);
}
false
```

Corrida mental

`contains(&n)` presta el elemento (`&i32`) para consultar sin moverlo — el `Vec` original queda intacto.

`insert(*n)` copia el `i32` (Copy) al set; se podría acortar usando que `insert` devuelve `false` si ya existía.

Cheatsheet: `&T` para consultar · `i32` se copia al insertar